



Carrera:
Licenciatura en
Ing. En Programación y Web master

Docente:
L.I.S.C. Mario Alonso Segovia Gutiérrez

Materia:
Programación Avanzada

Alumno(s):
Pedro José Marín Acosta

Fecha:
9 de agosto del 2026

Patrones de diseño en el desarrollo de software

Cuadro Comparativo

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Singleton	Creacional	Evita Múltiples Instancias que deben ser únicas. Centraliza el acceso a un recurso compartido. Previene Inconsistencias en entornos concurrentes.	Garantizar una única instancia de una clase. Ofrecer acceso controlado y global a esa instancia.	Constructor privado. Referencia estática. Método estático de acceso. Control de concurrencias.	Control centralizado de recursos. Ahorro de memoria al evitar instancias duplicadas. Facilita acceso global a datos o servicios. Sencillo de implementar en la mayoría de lenguajes	Introduce estado global, dificulta pruebas unitarias. Acoplamiento fuerte entre clases. Dificil de extender o sustituir en sistemas grandes. Considerado un antipatrón si se usa indiscriminadamente.	Gestión de configuración. Logger(único o archivo de logs). Gestión de conexiones. Drivers de hardware o recursos exclusivos.	Control de recursos compartidos como un logger o configuración global.	Baja en código, moderada en arquitectura

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Factor y method	Creacional	Evita el acoplamiento rígido entre el cliente y clases concretas. Centraliza la lógica de creación de objetos, reduciendo duplicación y errores. Facilita la extensión: nuevas variantes de productos se agregan sin modificar código existente.	Definir una interfaz para crear objetos. Aplicar el principio abierto / cerrado.	Producto (interfaz/abstracta): define operaciones comunes. Productos concretos: implementaciones específicas. Creador (abstracto): declara el método fábrica. Creadores concretos: sobrescriben el método fábrica para devolver productos concretos.	Desacopla cliente y clases concretas. Facilita polimorfismo y extensibilidad. Permite personalizar la creación (configuración, validaciones). Cumple con SOLID (OCP, DIP)	Mayor complejidad estructural (más clases y jerarquías). Sobrecarga innecesaria si solo se requiere una creación simple. Puede ser confuso para proyectos pequeños.	Frameworks de UI: factorías que crean ventanas según la plataforma. Gestión de conexiones: distintos conectores (HTTP, FTP, SSH). Procesamiento de archivos: lectores/escritores para JSON, XML, CSV. Drivers y parsers: selección dinámica según contexto.	Creación flexible de objetos	Moderada: requiere más clases que Singleton. Conceptual media-alta: útil en arquitecturas extensibles, pero innecesario en sistemas simples. Nivel de dificultad: intermedio, ideal para frameworks y librerías.

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Abstract Factory	Creacional	Necesidad de crear familias de objetos relacionados sin acoplar el código a clases concretas. Evita que el cliente tenga que decidir qué clase instanciar cuando hay múltiples variantes	Proporcionar una interfaz para crear familias de objetos relacionados o dependientes, sin especificar sus clases concretas. Permitir que el sistema sea extensible y configurable con distintas variantes.	Abstract Factory (interfaz): declara métodos para crear productos relacionados. Concrete Factories: implementan la interfaz y devuelven productos específicos. Productos abstractos: definen interfaces comunes. Productos concretos: implementaciones específicas de cada familia.	Desacopla el cliente de clases concretas. Garantiza consistencia entre objetos de la misma familia. Facilita la extensión con nuevas familias de productos.	Mayor complejidad que Factory Method (más interfaces y clases). Difícil de escalar si hay muchas variantes. Sobrecarga innecesaria en sistemas pequeños.	Frameworks de UI multiplataforma. Sistemas de persistencia (MySQL/PostgreSQL/SQLite). Aplicaciones con temas o estilos. Drivers de hardware con distintas implementaciones.	Crear familias de objetos relacionados (productos A y B) sin acoplar el cliente a clases concretas.	Moderada – Alta: estructura más extensa, ideal para frameworks o sistemas multiplataforma

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Builder	Creacional	Evita Constructores con demasiados parámetros. Elimina la necesidad de crear múltiples subclases para cada combinación de configuraciones. Evita el código duplicado centralizando la lógica de la construcción en un solo lugar.	Separar la construcción de un objeto complejo de su representación final, permitiendo crear diferentes variantes con el mismo proceso.	Builder (interfaz): define los pasos de construcción. Concrete Builders: implementan los pasos para crear distintas representaciones. Producto: el objeto complejo que se construye. Director (opcional): coordina el orden de los pasos de construcción.	Construcción paso a paso, flexible. Permite múltiples representaciones del mismo objeto. Evita constructores con parámetros innecesarios. Facilita el encadenamiento de métodos.	Más clases y estructura que otros patrones simples. Puede ser sobrecarga en objetos sencillos.	Objetos con muchas configuraciones opcionales (ej. formularios, reportes, documentos). Construcción de estructuras complejas. Exportadores: generar distintos formatos (PDF, HTML, JSON) con el mismo proceso.	Ejemplo builder.	Estructura: moderada – alta. Conceptual: media. Implementación: moderada con soporte para encadenamiento. Buena escalabilidad. Nivel de dificultad intermedio.

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Adaptee	Estructural	Incompatibilidad de interfaces. Reutilización de código existente. Integración con librerías externas: facilita conectar sistemas con APIs o servicios de terceros.	Convertir la interfaz de una clase en otra que el cliente espera, permitiendo que clases incompatibles trabajen juntas.	Target (interfaz esperada): define la interfaz que el cliente usa. Adaptee (clase existente): clase con una interfaz diferente. Adapter: traduce las llamadas del cliente a la interfaz del Adaptee. Cliente: usa el Target sin preocuparse de las diferencias internas.	Reutiliza clases existentes sin modificarlas. Facilita integración con librerías externas. Desacopla cliente y clase adaptada.	Añade una capa extra de complejidad. Puede generar sobrecarga si se abusa del patrón. No soluciona problemas de rendimiento inherentes al Adaptee.	Integrar APIs externas. Compatibilizar librerías antiguas con nuevas interfaces. Unificar formatos de datos (ej. JSON ↔ XML). Adaptar drivers o servicios con diferentes protocolos.	Ejemplo Adapter.	Estructural: baja – moderada. Conceptual: sencilla, fácil de entender. Implementación: directa, pocas clases. Buena escalabilidad para integrar múltiples librerías o servicios. Nivel de dificultad: bajo – intermedio.

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Facade	Estructural	Complejidad excesiva: cuando un sistema tiene demasiadas clases y subsistemas, el cliente se ve obligado a conocerlos todos. Acoplamiento fuerte: el cliente depende directamente de múltiples clases internas. Dificultad de uso: interfaces poco amigables para tareas comunes.	Proporcionar una interfaz simplificada y unificada para un conjunto complejo de clases o subsistemas. Facilitar el uso del sistema ocultando la complejidad interna.	Subsistemas: clases internas con lógica compleja. Facade: clase que expone métodos simples y delega llamadas a los subsistemas. Cliente: interactúa solo con el Facade, sin conocer los detalles internos.	Reduce la complejidad para el cliente. Desacopla el código cliente de los subsistemas. Mejora la legibilidad y mantenibilidad. Facilita la integración con sistemas externos.	Puede ocultar demasiada funcionalidad, limitando el acceso a operaciones avanzadas. Si el Facade crece demasiado, puede convertirse en un "Dios objeto".	Frameworks: ofrecer una API simple sobre subsistemas complejos. Sistemas de librerías: simplificar el acceso a múltiples módulos. Aplicaciones grandes: centralizar operaciones comunes (ej. inicialización, configuración).	Ejemplo Facade	

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Decorator	Estructural	Herencia rígida: cuando necesitas añadir funcionalidades a una clase sin crear múltiples subclases. Combinaciones explosivas: evita tener que definir todas las variantes posibles de una clase. Extensión dinámica: permite agregar responsabilidades en tiempo de ejecución.	Agregar funcionalidades adicionales a un objeto de manera dinámica, envolviéndolo en clases decoradoras sin modificar su código original.	Componente (interfaz): define la operación estándar. Componente concreto: implementación básica. Decorador abstracto: mantiene referencia al componente y delega operaciones. Decoradores concretos: añaden nuevas responsabilidades antes o después de delegar.	Extiende funcionalidades sin modificar clases existentes. Permite combinar decoradores de forma flexible. Favorece el principio de composición sobre herencia.	Puede generar demasiados objetos pequeños. La depuración se complica si hay muchos decoradores anidados.	Streams de entrada/salida (ej. añadir compresión, cifrado, buffering). Interfaz gráfica (añadir bordes, sombras, scrollbars). Logs y monitoreo.	Ejemplo Decorador.	Estructural: Moderada. Conceptual: media, fácil de entender si se piensa en “envolver objetos”. Implementación: moderada, con múltiples clases pequeñas. Muy buena escalabilidad. permitiendo infinitas combinaciones de funcionalidades. Nivel de dificultad: intermedio.

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Observer	Comportamiento	Dependencias rígidas: cuando múltiples objetos necesitan reaccionar a cambios en otro objeto. Acoplamiento fuerte: evita que el sujeto tenga que conocer directamente a todos sus observadores. Notificaciones manuales: elimina la necesidad de actualizar objetos dependientes uno por uno.	Definir una relación de suscripción entre objetos, de modo que cuando un objeto cambie de estado, todos sus dependientes sean notificados automáticamente.	Subject (sujeto): mantiene una lista de observadores y notifica cambios. Observer (observador): define la interfaz de actualización. Concrete Subject: implementa la lógica y dispara notificaciones. Concrete Observers: reaccionan a las actualizaciones.	Desacopla el sujeto de sus observadores. Permite añadir o quitar observadores dinámicamente. Facilita la comunicación uno-a-muchos.	Puede generar demasiadas notificaciones si no se controla. Difícil de depurar cuando hay muchos observadores encadenados. Riesgo de dependencias circulares.	Interfaces gráficas: actualizar vistas cuando cambia el modelo. Eventos en sistemas: listeners en frameworks. Notificaciones: sistemas de mensajería, logging, monitoreo. Bases de datos: triggers que reaccionan a cambios.	Ejemplo Observer.	Estructural: moderada (requiere Subject, Observer y sus implementaciones). Conceptual: media, fácil de entender como "suscripción a eventos". Implementación: moderada, con listas de observadores y métodos de notificación. Buena escalabilidad: soporta múltiples observadores dinámicos. Nivel de dificultad: intermedio.

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Strategy	Comportamiento	Condicionales extensos: evita tener múltiples if/else o switch para seleccionar comportamientos. Rigidez en algoritmos: permite cambiar la lógica sin modificar el cliente. Falta de flexibilidad: facilita intercambiar algoritmos en tiempo de ejecución.	Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables, permitiendo que el cliente use diferentes estrategias sin acoplarse a ellas.	Strategy (interfaz): define el método común para los algoritmos. Concrete Strategies: implementan diferentes variantes del algoritmo. Context: mantiene una referencia a la estrategia y la ejecuta. Cliente: selecciona la estrategia adecuada según necesidad.	Elimina condicionales complejos. Permite cambiar algoritmos en tiempo de ejecución. Favorece el principio de abierto/cerrado (OCP). Facilita pruebas unitarias de cada estrategia.	Más clases y objetos que una solución simple. El cliente debe conocer las diferencias entre estrategias. Puede ser sobrecarga en casos triviales.	Sistemas de pago: elegir entre PayPal, tarjeta, transferencia. Compresión de archivos: distintos algoritmos (ZIP, RAR, GZIP). Ordenamiento: distintos métodos (rápido, burbuja, mergesort). Autenticación: diferentes estrategias (OAuth, JWT, LDAP).	Ejemplo Strategy.	

Patrón	Categoría	Problemas que resuelve	Objetivo Principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo real	Complejidad
Command	Comportamiento	Acoplamiento fuerte: evita que el cliente conozca directamente cómo se ejecuta una acción. Historial y deshacer: permite registrar operaciones y revertirlas. Flexibilidad en ejecución: encapsula solicitudes como objetos, facilitando su almacenamiento, cola o ejecución diferida.	Encapsular una solicitud como un objeto, permitiendo parametrizar clientes con diferentes operaciones, encolar solicitudes y soportar operaciones como deshacer.	Command (interfaz): declara el método execute(). Concrete Commands: implementan la acción específica, llamando al receptor. Receiver: contiene la lógica real de la operación. Invoker: solicita la ejecución del comando. Cliente: crea comandos y los asigna al invocador.	Desacopla el emisor de la acción del receptor. Permite implementar fácilmente deshacer/rehacer. Facilita la creación de colas de comandos y macros. Favorece la extensibilidad: nuevos comandos sin modificar el invocador.	Más clases y objetos que una solución directa. Puede ser sobrecargada en operaciones simples.	Sistemas de UI: botones y menús que ejecutan acciones. Editor de texto: soporte para deshacer/rehacer. Colas de tareas: ejecutar comandos en diferido. Automatización: macros que agrupan múltiples comandos.	Ejemplo Command	Estructural: moderada (Command, Concrete Commands, Receiver, Invoker). Conceptual: media, fácil de entender como "encapsular acciones". Implementación: moderada, requiere varias clases pequeñas. Muy Buena escalabilidad: soporta nuevas operaciones sin modificar el invocador. Nivel de dificultad: intermedio.

Singleton: Control de recursos compartidos como un logger o configuración global.

```
<?php
class Singleton {
    private static $instances = [];

    protected function __construct() {}
    protected function __clone() {}
    public function __wakeup() {
        throw new \Exception("Cannot unserialize a singleton.");
    }

    public static function getInstance(): Singleton {
        $cls = static::class;
        if (!isset(self::$instances[$cls])) {
            self::$instances[$cls] = new static();
        }
        return self::$instances[$cls];
    }

    public function someBusinessLogic() {
        // lógica de negocio
    }
}

// Cliente
$s1 = Singleton::getInstance();
$s2 = Singleton::getInstance();
echo $s1 === $s2 ? "Singleton funciona" : "Singleton falló";
```

Creación flexible de objetos factory method

```
<?php
// Producto
interface Product {
    public function operation(): string;
}

// Productos concretos
class ConcreteProduct1 implements Product {
    public function operation(): string { return "{Result of Product1}"; }
}
class ConcreteProduct2 implements Product {
    public function operation(): string { return "{Result of Product2}"; }
}

// Creador abstracto
abstract class Creator {
    abstract public function factoryMethod(): Product;

    public function someOperation(): string {
        $product = $this->factoryMethod();
        return "Creator: worked with " . $product->operation();
    }
}

// Creadores concretos
class ConcreteCreator1 extends Creator {
    public function factoryMethod(): Product { return new ConcreteProduct1(); }
}
class ConcreteCreator2 extends Creator {
    public function factoryMethod(): Product { return new ConcreteProduct2(); }
}

// Cliente
echo (new ConcreteCreator1())->someOperation();
echo (new ConcreteCreator2())->someOperation();
```

Crear familias de objetos relacionados (productos A y B) sin acoplar el cliente a clases concretas. Abstract factory.

```
<?php
// Abstract Factory
interface AbstractFactory {
    public function createProductA(): AbstractProductA;
    public function createProductB(): AbstractProductB;
}

// Productos abstractos
interface AbstractProductA { public function usefulFunctionA(): string; }
interface AbstractProductB { public function usefulFunctionB(): string; }

// Productos concretos (variante 1)
class ProductA1 implements AbstractProductA {
    public function usefulFunctionA(): string { return "Producto A1"; }
}
class ProductB1 implements AbstractProductB {
    public function usefulFunctionB(): string { return "Producto B1"; }
}

// Productos concretos (variante 2)
class ProductA2 implements AbstractProductA {
    public function usefulFunctionA(): string { return "Producto A2"; }
}
class ProductB2 implements AbstractProductB {
    public function usefulFunctionB(): string { return "Producto B2"; }
}

// Factories concretas
class ConcreteFactory1 implements AbstractFactory {
    public function createProductA(): AbstractProductA { return new ProductA1(); }
    public function createProductB(): AbstractProductB { return new ProductB1(); }
}
class ConcreteFactory2 implements AbstractFactory {
    public function createProductA(): AbstractProductA { return new ProductA2(); }
    public function createProductB(): AbstractProductB { return new ProductB2(); }
}

// Cliente
function clientCode(AbstractFactory $factory) {
    $a = $factory->createProductA();
    $b = $factory->createProductB();
    echo $a->usefulFunctionA() . " + " . $b->usefulFunctionB();
}

clientCode(new ConcreteFactory1()); // Producto A1 + Producto B1
clientCode(new ConcreteFactory2()); // Producto A2 + Producto B2
```

Ejemplo builder: <?php ?>

```
<?php
interface Builder {
    public function producePartA(): void;
    public function producePartB(): void;
}

class ConcreteBuilder1 implements Builder {
    private $product;
    public function __construct() { $this->reset(); }
    public function reset(): void { $this->product = new
Product1(); }
    public function producePartA(): void { $this->product->parts[]
= "PartA1"; }
    public function producePartB(): void { $this->product->parts[]
= "PartB1"; }
    public function getResult(): Product1 { return $this->product; }
}

class Product1 {
    public $parts = [];
    public function listParts() { echo implode(", ", $this->parts); }
}

// Uso real
$builder = new ConcreteBuilder1();
$builder->producePartA();
$builder->producePartB();
$product = $builder->getResult();
$product->listParts(); // PartA1, PartB1
```

Ejemplo Adapter <?php ?>

```
<?php
// Target
class Target {
    public function request(): string {
        return "Target: comportamiento estándar";
    }
}

// Adaptee (interfaz diferente)
class Adaptee {
    public function specificRequest(): string {
        return "Adaptee: comportamiento específico";
    }
}

// Adapter
class Adapter extends Target {
    private $adaptee;
    public function __construct(Adaptee $adaptee) {
        $this->adaptee = $adaptee;
    }
    public function request(): string {
        return "Adapter: " . $this->adaptee->specificRequest();
    }
}

// Cliente
$adaptee = new Adaptee();
$adapter = new Adapter($adaptee);
echo $adapter->request(); // Adapter: Adaptee: comportamiento
específico
```

Ejemplo Facade <?php ?>

```
<?php
// Subsistemas
class SubsystemA {
    public function operationA(): string { return "A"; }
}
class SubsystemB {
    public function operationB(): string { return "B"; }
}

// Facade
class Facade {
    private $a;
    private $b;
    public function __construct(SubsystemA $a, SubsystemB $b) {
        $this->a = $a;
        $this->b = $b;
    }
    public function operation(): string {
        return $this->a->operationA() . " + " . $this->b->operationB();
    }
}

// Cliente
$a = new SubsystemA();
$b = new SubsystemB();
$facade = new Facade($a, $b);
echo $facade->operation(); // A + B
```

Ejemplo Decorator <?php ?>

```
<?php
// Componente
interface Component {
    public function operation(): string;
}

// Componente concreto
class ConcreteComponent implements Component {
    public function operation(): string { return "Base"; }
}

// Decorador abstracto
class Decorator implements Component {
    protected $component;
    public function __construct(Component $component) { $this->component = $component; }
    public function operation(): string { return $this->component->operation(); }
}

// Decoradores concretos
class DecoratorA extends Decorator {
    public function operation(): string { return "A(" . parent::operation() . ")"; }
}
class DecoratorB extends Decorator {
    public function operation(): string { return "B(" . parent::operation() . ")"; }
}

// Cliente
$base = new ConcreteComponent();
echo (new DecoratorA($base))->operation(); // A(Base)
echo (new DecoratorB(new DecoratorA($base)))->operation(); // B(A(Base))
```

Ejemplo observer <?php ?>

```
<?php
// Subject
interface Subject {
    public function attach(Observer $o): void;
    public function detach(Observer $o): void;
    public function notify(): void;
}

class ConcreteSubject implements Subject {
    private $observers = [];
    public $state;

    public function attach(Observer $o): void { $this->observers[] = $o; }
    public function detach(Observer $o): void { $this->observers = array_diff($this->observers, [$o]); }
    public function notify(): void {
        foreach ($this->observers as $o) { $o->update($this); }
    }
    public function changeState($val): void {
        $this->state = $val;
        $this->notify();
    }
}

// Observer
interface Observer { public function update(Subject $s): void; }

// Observadores concretos
class ConcreteObserverA implements Observer {
    public function update(Subject $s): void { echo "ObserverA: nuevo estado = $s->state\n"; }
}
class ConcreteObserverB implements Observer {
    public function update(Subject $s): void { echo "ObserverB: reaccionando al estado $s->state\n"; }
}

// Cliente
$subject = new ConcreteSubject();
$subject->attach(new ConcreteObserverA());
$subject->attach(new ConcreteObserverB());

$subject->changeState("Activo");
// ObserverA: nuevo estado = Activo
// ObserverB: reaccionando al estado Activo
```

Ejemplo Strategy <?php ?>

```
<?php
// Strategy
interface Strategy {
    public function doAlgorithm(array $data): array;
}

// Concrete Strategies
class ConcreteStrategyA implements Strategy {
    public function doAlgorithm(array $data): array {
        sort($data);
        return $data;
    }
}
class ConcreteStrategyB implements Strategy {
    public function doAlgorithm(array $data): array {
        rsort($data);
        return $data;
    }
}

// Context
class Context {
    private $strategy;
    public function __construct(Strategy $strategy) { $this->strategy = $strategy; }
    public function setStrategy(Strategy $strategy) { $this->strategy = $strategy; }
    public function execute(array $data): void {
        $result = $this->strategy->doAlgorithm($data);
        echo implode(", ", $result) . "\n";
    }
}

// Cliente
$data = ["a", "b", "c"];
$context = new Context(new ConcreteStrategyA());
$context->execute($data); // a, b, c

$context->setStrategy(new ConcreteStrategyB());
$context->execute($data); // c, b, a
```

Ejemplo Command <?php ?>

```
<?php
// Command
interface Command { public function execute(): void; }

// Receiver
class Receiver {
    public function actionA() { echo "Acción A\n"; }
    public function actionB() { echo "Acción B\n"; }
}

// Concrete Commands
class CommandA implements Command {
    private $receiver;
    public function __construct(Receiver $r) { $this->receiver = $r; }
    public function execute(): void { $this->receiver->actionA(); }
}
class CommandB implements Command {
    private $receiver;
    public function __construct(Receiver $r) { $this->receiver = $r; }
    public function execute(): void { $this->receiver->actionB(); }
}

// Invoker
class Invoker {
    private $command;
    public function setCommand(Command $c) { $this->command = $c; }
    public function run(): void { $this->command->execute(); }
}

// Cliente
$r = new Receiver();
$invoker = new Invoker();

$invoker->setCommand(new CommandA($r));
$invoker->run(); // Acción A

$invoker->setCommand(new CommandB($r));
$invoker->run(); // Acción B
```


Fuentes

Refactoring Guru. (s.f.). Singleton en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/singleton/php>

Refactoring Guru. (s.f.). Factory Method en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/factory-method/php>

Refactoring Guru. (s.f.). Abstract Factory en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/abstract-factory/php>

Refactoring Guru. (s.f.). Builder en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/builder/php>

Refactoring Guru. (s.f.). Adapter en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/adapter/php>

Refactoring Guru. (s.f.). Facade en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/facade/php>

Refactoring Guru. (s.f.). Decorator en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/decorator/php>

Refactoring Guru. (s.f.). Observer en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/observer/php>

Refactoring Guru. (s.f.). Strategy en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/strategy/php>

Refactoring Guru. (s.f.). Command en PHP. Refactoring Guru. Recuperado el 9 de agosto de 2026, de <https://refactoring.guru/es/design-patterns/command/php>